



**JAX: Functional  
Transformations for  
High-Performance  
ML (2018) -  
developed by  
Google**

- Transform numerical Python/NumPy functions
- Functional programming paradigm; immutable arrays
- XLA (accelerated linear algebra)-backed just-in-time compilation for CPUs/GPUs/TPUs

**Why It Matters:**

- Research velocity with concise, pure functions
- Performance: kernel fusion, layout optimization, device placement via XLA
- Scales from single device to multi-accelerator setups (TPUs/GPUs)

**Just-in-Time (JIT) compilation** is a method for executing computer code that translates it into native machine code at runtime. It is a hybrid approach that combines the best aspects of traditional interpreters (flexibility) and static, ahead-of-time (AOT) compilers (performance).

**XLA, which stands for Accelerated Linear Algebra**, is an open-source compiler infrastructure designed to improve the performance of machine learning models and other numerical computations, particularly those involving large-scale array operations.

# Framework Comparison Table

|                           | TensorFlow                       | PyTorch          | JAX                        |
|---------------------------|----------------------------------|------------------|----------------------------|
| Graph Type                | Static (1.x), Dynamic (2.x)      | Dynamic          | Functional transformations |
| Programming Model         | Imperative (2.x), Symbolic (1.x) | Imperative       | Functional                 |
| Core Data Structure       | Tensor (mutable)                 | Tensor (mutable) | Array (immutable)          |
| Execution Mode            | Eager (2.x default), Graph       | Eager            | Just-in-time compilation   |
| Automatic Differentiation | Reverse mode                     | Reverse mode     | Forward and Reverse mode   |
| Hardware Acceleration     | CPU, GPU, TPU                    | CPU, GPU         | CPU, GPU, TPU              |

**Eager execution** in TensorFlow is an imperative programming environment where operations are evaluated immediately as they are called, rather than building a computational graph to be run later. This contrasts with TensorFlow's traditional graph mode, where operations are first added to a symbolic graph and then executed in a TensorFlow session.

# Hardware Impact on Framework Design

## GPU Revolution (NVIDIA CUDA - 2007)

- **General-purpose computing on GPUs**
- **Orders of magnitude speedup** for matrix operations
- **Parallel processing** of thousands of elements simultaneously
- **Specialized Accelerators**



## Google TPUs (2016) - purpose-built for tensor operations

- **Systolic array architectures** - efficient for matrix multiplication
- **Mobile accelerators** - Apple Neural Engine, Qualcomm NPUs

## Impact on Frameworks:

- **Specialized compilation strategies**
- **New optimization opportunities**
- **Hardware-specific operator implementations**



Google TPU (Image source: <https://www.servethehome.com>)

# Quiz

1) Why do deep learning workloads map well to GPUs?

- A. GPUs have larger caches than CPUs
- B. GPUs excel at massively parallel dense linear algebra like GEMMs/convolutions
- C. GPUs run higher clock speeds than CPUs
- D. GPUs avoid using memory entirely

2) In data-parallel training, what communication step typically synchronizes model replicas each iteration?

- A. Parameter server RPCs
- B. All-to-all exchange of activations
- C. All-reduce of gradients
- D. Broadcasting the dataset

3) Which statement best describes JAX's programming model?

- A. Imperative mutable tensors with static graphs
- B. Pure functions over immutable arrays transformed by jit/grad/vmap/pjit
- C. Graphs defined in a separate configuration language
- D. Only forward-mode differentiation is supported

4) What benefit does kernel fusion provide on accelerators?

- A. Increases model accuracy
- B. Reduces op launch overhead and memory traffic by combining ops
- C. Eliminates the need for backpropagation
- D. Doubles device memory capacity

5) When scaling batch size across more GPUs, which training adjustment is commonly recommended?

- A. Decrease learning rate proportionally to batch size
- B. Keep learning rate fixed regardless of batch
- C. Use the linear scaling rule and warmup the learning rate
- D. Disable mixed precision to stabilize training

# Week 2 Overview

## Automatic Differentiation: The Engine of Modern ML

### Learning Objectives:

- Master forward and reverse mode automatic differentiation algorithms
- Analyze memory and computational trade-offs between AD modes
- Evaluate AD implementation strategies for resource-constrained environments
- Design efficient gradient computation systems for edge and embedded deployment

### Required Readings:

- Book 1 - Volume 1: Neural Computation
- Book 1 - Volume 1: ML Frameworks
- Book 1 - Volume 1: Model Training
- Book 2: Chapter 3: Getting Up to Speed on Machine Learning



NETFLIX

